

JUNIT :

Exercise 1: Setting Up JUnit

1.setting up a pom.xml

2. CALCULATOR .JAVA

```
package com.cognizant.calculator;
```

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
}
```

3. CALCULATORTEST.JAVA

```
package com.cognizant.calculator;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
import org.junit.jupiter.api.Test;
```

```
public class CalculatorTest {  
  
    @Test  
    void testAddition() {  
  
        Calculator calculator = new Calculator();  
  
        int result = calculator.add(10, 20);  
  
        assertEquals(30, result);  
  
    }  
}
```

OUTPUT :

Tests run: 1

Failures: 0

2. ASSERTION IN JUNIT

1.ASSERTEQUALS

```
package com.cognizant.calculator;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class CalculatorTest {

    @Test
    void testAddition() {

        Calculator c = new Calculator();

        int result = c.add(10,20);

        assertEquals(30,result);

    }

}
```

2.AssertTrue()

```
@Test
void testTrue(){

    int age = 20;

    assertTrue(age>=18);

}
```

3.AssertNotNull()

```
@Test
void testNotNull(){

    String name = "Thanisha";

    assertNotNull(name);

}
```

4. Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

CALCULATORAAA TEST :

```
package com.cognizant.exercise4;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.*;

public class CalculatorAAATest {

    private CalculatorAAA calculator;

    @BeforeEach
    void setUp() {
        System.out.println("Creating Calculator Object...");
        calculator = new CalculatorAAA();
    }

    @AfterEach
    void tearDown() {
        System.out.println("Test Completed.");
    }

    @Test
    void testAddition() {

        // Arrange
        int a = 10;
        int b = 20;

        // Act
        int result = calculator.add(a, b);

        // Assert
        assertEquals(30, result);
    }

    @Test
    void testSubtraction() {

        // Arrange
        int a = 30;
```

```
int b = 10;
```

```
// Act
```

```
int result = calculator.subtract(a, b);
```

```
// Assert
```

```
assertEquals(20, result);
```

```
}
```

```
}
```

CALCULATORAAA .JAVA :

```
package com.cognizant.exercise4;
```

```
public class CalculatorAAA {
```

```
    public int add(int a, int b) {
```

```
        return a + b;
```

```
    }
```

```
    public int subtract(int a, int b) {
```

```
        return a - b;
```

```
    }
```

```
    public int multiply(int a, int b) {
```

```
        return a * b;
```

```
    }
```

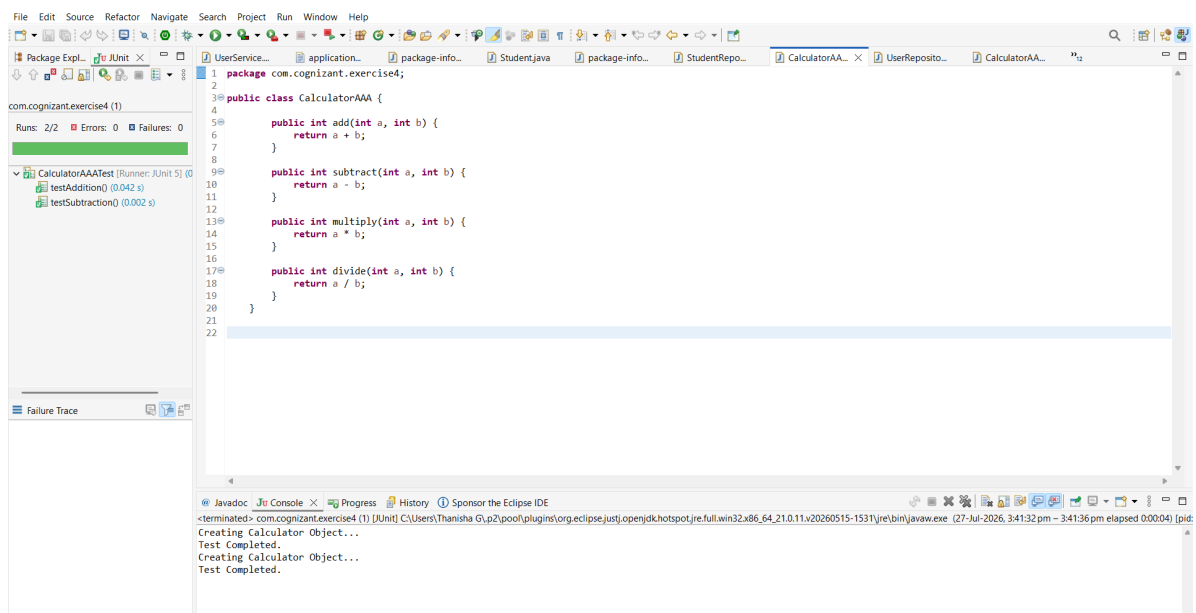
```
    public int divide(int a, int b) {
```

```
        return a / b;
```

```
    }
```

```
}
```

OUTPUT :



MOCKITO :

EXERCISE 1 : MOCKING AND STUBBING :

UserRepository.java

```
package com.cognizant.mockito1;
```

```
public interface UserRepository {
```

```
    String getUserName();
```

```
}
```

UserService.java

```
package com.cognizant.mockito1;
```

```
public class UserService {
```

```
    private UserRepository repository;
```

```
    public UserService(UserRepository repository) {
```

```
        this.repository = repository;
```

```
}
```

```
    public String getUser() {
```

```
        return repository.getUserName();
```

```
}
```

```
}
```

UserServiceTest.java

```
package com.cognizant.mockito1;
```

```
import static org.junit.jupiter.api.Assertions.assertEquals;
```

```
import static org.mockito.Mockito.*;
```

```
import org.junit.jupiter.api.Test;
```

```
public class UserServiceTest {
```

```
    @Test
```

```
    void testGetUser() {
```

```

// Create Mock
UserRepository repository = mock(UserRepository.class);

// Stubbing
when(repository.getUserName()).thenReturn("Thanisha");

// Inject Mock
UserService service = new UserService(repository);

// Call Method
String result = service.getUser();

// Verify Output
assertEquals("Thanisha", result);

}

}

```

OUTPUT :

